

ECE 374 B: Algorithms and Models of Computation, Summer 2026

Midterm 3 – August 04, 2025

-
- You will have 75 minutes (1.25 hours) to solve all the problems. Most have multiple parts. Don't spend too much time on questions you don't understand and focus on answering as much as you can!
 - **BUDGET YOUR TIME WISELY.** I highly recommend working on the questions you know first and the questions you need to think about second.
 - No resources are allowed for use during the exam except a multi-page cheatsheet and scratch paper on the back of the exam. **Do not tear out the cheatsheet or the scratch paper!** It messes with the auto-scanner.
 - You should write your answers *completely* in the space given for the question. We will not grade parts of any answer written outside of the designated space.
 - Please *use a dark-colored pen* unless you are *absolutely* sure your pencil writing is forceful enough to be legible when scanned. We reserve the right to take off points if we have difficulty reading the uploaded document.
 - Unless otherwise stated, assume $P \neq NP$.
 - Assume that whenever the word "reduction" is used, we mean a (not necessarily polynomial-time) *mapping/many-one* reduction.
 - You can only refer to the cheat sheet content as a black box.
 - **Don't cheat.** If we catch you, you will get an F in the course.
 - **Good luck!**
-

Name: _____

NetID: _____

Date: _____

1 Short Answer I - 28 points

For each of the problems circle *true* if the statement is *always* true, circle *false* otherwise. There is no partial credit for these questions.

- (a) NP stands for “non-polynomial” time.

Solution: True ☒ False ☐
It stands for “nondeterministic polynomial” time. ■

- (b) Every problem in P is also in NP.

Solution: True ☒ False ☐
$P \subseteq NP$. ■

- (c) If L is an NP-complete language and $L \in P$, then $P = NP$

Solution: True ☒ False ☐
$L \in NPC \Rightarrow L \in NP\text{-Hard} \wedge L \in NP \wedge L \in P \Rightarrow \exists L \in NP\text{-Hard} \cap P \Rightarrow P = NP$ Or in plain english: NP-complete means all NP problems are reducible in polynomial time to a problem X. If X is solvable in polynomial time, then it means there is a polynomial time algorithm for all problems in NP. ■

- (d) There exists a polynomial time reduction from every NP-hard problem to every problem in P.

Solution: True ☐ False ☒
See above explanation. This would imply $P=NP$ which we generally don't assume. ■

- (e) Every NP-hard problem must be decidable.

Solution: True ☐ False ☒
NP-hard problems could be undecidable. ■

- (f) If a problem is NP-complete, then it must be a decision problem.

Solution: True ☒ False ☐
Under the standard definitions, NP is a class of decision problems, so an NP-complete problem must be a decision problem. ■

- (g) If there is a polynomial time reduction from problem A to problem B and B is in NP, then A must also be in NP.

Solution: ☒ True ☐ False

The reduction gives gives a upperbound of the time complexity. ■

- (h) If a problem can be solved in polynomial space, then it is also solvable in polynomial time.

Solution: ☐ True ☒ False

Plenty of problems require less space complexity than time complexity. ■

- (i) If a problem is solvable in polynomial time, then it must also be solvable in polynomial space.

Solution: ☒ True ☐ False

You can't need more space than time. You need that much time at minimum to just write to the storage! ■

- (j) There are problems (X) that are in NP for which the reduction $X \leq 3SAT$ cannot be shown.

Solution: ☐ True ☒ False

The definition of NP-complete is all problems are reducible to that problem. ■

- (k) A problem that requires exponential time must also require exponential space.

Solution: ☐ True ☒ False

A computation may reuse a polynomial amount of memory for exponentially many steps. Large time complexity does not necessarily imply equally large space complexity. ■

- (l) All NP-hard problems must have yes/no outputs.

Solution: ☐ True ☒ False

There are more problems than just decision problems. Optimization problems (longest/smallest/minimum) are still NP hard but don't have a true/false output. ■

- (m) If L is decidable, then $\bar{L}$ must be also be decidable:

Solution: ☒ True ☐ False

There are more problems than just decision problems. Optimization problems (longest/smallest/minimum) are still NP hard but don't have a true/false output. ■

- (n) If L is turing-recognizable but not decidable, then $\bar{L}$ must be Turing-unrecognizable:

Solution: ☒ True ☐ False

There are more problems than just decision problems. Optimization problems (longest/smallest/minimum) are still NP hard but don't have a true/false output. ■

2 Short Answer III - 12 points

Assuming the reductions below can be proven, circle all the classes that the problem X may belong to (no explanation, this is just multiple choice) :

- $CLIQUE \leq_p X$

Solution: P NP NP-hard NP-complete

decidable undecidable

■

- $X \leq_p \text{Longest_increasing_subsequence}$

Solution: P NP NP-hard NP-complete

decidable undecidable

■

- $A_{TM} \Rightarrow X$

Solution: P NP NP-hard NP-complete

decidable undecidable

■

3 Classification I (P/NP) - 12 points

Is the following problem in P, NP, or some combinations of complexity classes? For each of the following problems, circle all the complexity classes that problem belongs to. Whatever class it is in, prove it!

FewerThanKPath (FTKP) problem - You are given a directed, acyclic path and need to find if there is a path from s to t with a weight less than k .

- Input: A weighted (positive & negative) directed acyclic graph G , start/end nodes s/t , and a number k .
- Output: True if there exists a path from s to t with a weight less than k . False otherwise.

Which of the following complexity classes does this problem belong to? Circle **all** that apply:

Solution:

☒ P

☒ NP

☐ NP-hard

☐ NP-complete

The problem belongs to P and therefore also belongs to NP.

The DAG stuff is largely a red herring. Run Bellman Ford's algorithm from s to compute the shortest-path distance $d(s, t)$.

Return True if and only if both distances are finite and

$$d(s, r) + d(r, t) \leq k.$$

Every directed path from s to t that passes through r consists of an s -to- r portion followed by an r -to- t portion. Therefore, the minimum possible weight of such a path is

$$d(s, r) + d(r, t).$$

Because all edge weights are nonnegative, Dijkstra's algorithm correctly computes both shortest-path distances. Using a binary heap, the two runs take

$$O((|V| + |E|) \log |V|)$$

time in total.

Therefore,

$$\text{LCPC} \in \text{P}.$$

Since

$$\text{P} \subseteq \text{NP},$$

we also have

$$\text{LCPC} \in \text{NP}.$$

Thus, the correct choices are

☒ P

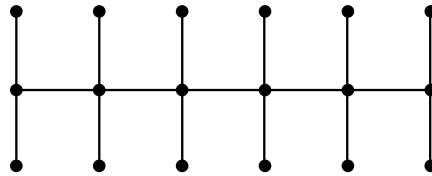
☒ NP.

We cannot conclude that the problem is NP-hard or NP-complete unless $P = NP$. ■

4 Classification II (P/NP) - 12 points

Is the following problem in P, NP, or some combinations of complexity classes? For each of the following problems, circle all the complexity classes that problem belongs to. Whatever class it is in, prove it!

A centipede is an undirected graph formed by a path of length k with two edges (legs) attached to each node on the path as shown in the below figure. Hence, the centipede graph has $3k$ vertices.



CENTIPEDE graph with $k = 6$

The CENTIPEDE problem asks whether there exists a centipede subgraph of $3k$ vertices :

- Input: A undirected graph $G = V, E$ and integer k
- Output: True if there does exist a CENTIPEDE graph. False otherwise.

Which of the following complexity classes does this problem belong to? Circle **all** that apply:

P NP NP-hard NP-complete

Solution: P

NP

NP-hard

NP-complete

NP: All we need to do is show a certificate and certifier:

- **Certificate:** A tuple list that denote the vertices along the spine and the vertices that are “feet”.
- **Certifier:** Need to make sure no vertex is used more than once and that a edge exists between every consecutive vertex in the spine and every spine vertex and it's feet. This can be done in linear time whihc makes the problem **NP**.

NP-hard: We reduce from Hamiltonian Path to Centipede. To do this reduction , we need to transform a graph that may have a HP into a graph that may have a CENTIPEDE iff the original graph had a HP. All we do is for every original vertex, we add two corresponding vertices to the graph and the attach them to the original vertex through some edges. If the originl graph had a HP, then this new graph has a CENTIPEDE of size $k = |V|$. This reduction shows that CENTIPEDE is NP-hard.

Therefore, we know this problem is: **NP-Complete: NP, NP-Hard.**



5 Classification I (Decidability) - 12 points

Are the following languages decidable? For each of the following languages,

- Circle one of "decidable" or "undecidable" to indicate your choice.
- If you choose "decidable", show your choice correct by describing an algorithm that decides that language. If you choose "undecidable", show your choice correct by giving a reduction proving its correctness.
- Regardless of your choice, explain *briefly* (i.e., in few sentences, diagrams, *clear* pseudo-code) why your choice is valid.

$\text{AcceptNoHomers}_{TM} = \{\langle M \rangle \mid M \text{ is a } TM \text{ and does not accept any strings that contain "Homer" as a substring}\}$

Solution: decidable

undecidable

DecideHalt($\langle M, w \rangle$):

Encode the following Turing machine D'oh!:

D'oh!(x):

run M on input w

return True

if DecideAcceptNoHomers($\langle \text{D'oh!} \rangle$)

return False

else

return True

If M halts on w , then $\langle \text{D'oh!} \rangle$ encodes a TM that accepts "Homer". So AcceptNoHomers returns False, and $\neg \text{AcceptNoHomers}$ returns True.

If M kicks the bucket on w , then $\langle \text{D'oh!} \rangle$ encodes a TM that doesn't accept "Homer". So AcceptNoHomers returns True, and $\neg \text{AcceptNoHomers}$ returns False.

12

Thus, if $\text{DecideAcceptNoHomers}$ existed, the algorithm above would decide Halt_{TM} . Since Halt_{TM} is undecidable, $\text{DecideAcceptNoHomers}$ cannot exist. Therefore, $\text{AcceptNoHomers}_{TM}$ is undecidable. ■

6 Classification II (Decidability) - 12 points

Are the following languages decidable? For each of the following languages,

- Circle one of "decidable" or "undecidable" to indicate your choice.
- If you choose "decidable", show your choice correct by describing an algorithm that decides that language. If you choose "undecidable", show your choice correct by giving a reduction proving its correctness.
- Regardless of your choice, explain *briefly* (i.e., in few sentences, diagrams, *clear* pseudo-code) why your choice is valid.

$$\text{AcceptSlowly}_{TM} = \{ \langle M, w \rangle \mid M \text{ is a } TM \text{ and accepts } w \text{ in } 2^{|w|} \text{ steps.} \}$$

Solution: This problem is constrained because we only need to simulate $2^{|w|}$ steps. Yes it's a lot of steps but its not infinite so the problem is decidable.

☒ decidable

☐ undecidable

```
DecideInTime( $M, w$ ):
  for step from 1 to  $2^{|w|}$ 
    simulate one tape step of  $M$  on input  $w$ 
  if  $M$  is in an accept state
    return True
  return False
```

■

7 Classification (Mixed) - 12 points

Same type of problem as before, except now you have to choose between a mix of computational and algorithmic complexity classes.

In the **MinVertexCover problem**, you are given an undirected graph and asked to find the size of the smallest set of vertices that touches every edge.

- Input: An unweighted undirected graph $G = (V, E)$.
- Output: A value x denoting the minimum number of vertices needed to cover every edge in G .

Which of the following classes does this problem belong to? Whatever you choose, *succinctly* prove it!

Solution: P NP **NP-Hard** NP-complete **decidable** undecidable

Note: P and NP are defined over **decision problems**. This problem outputs a number.

NP-Hard: We reduce

$$\text{VertexCover} \leq_p \text{MinVertexCover}.$$

```
VertexCoverSolver( $G, k$ ):
   $x = \text{MinVertexCoverOracle}(G)$ 
  return  $x \leq k$ 
```

→ If G has a vertex cover of size at most k , then the minimum vertex-cover size x satisfies $x \leq k$, so the algorithm returns True.

← If G has no vertex cover of size at most k , then $x > k$, so the algorithm returns False.

Decidable:

```
BruteMinVertexCover( $G$ ):
  for  $k$  from 0 to  $|V|$ 
    for each subset  $S \subseteq V$  of size  $k$ 
      if every edge has at least one endpoint in  $S$ 
        return  $k$ 
```

The algorithm checks finitely many subsets and always halts. Therefore, MinVertexCover is decidable. ■

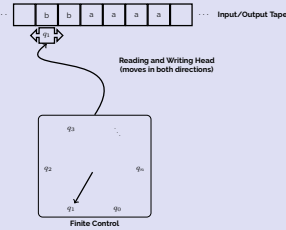
This page is for additional scratch work!

ECE 374 B Reductions, P/NP, and Decidability: Cheatsheet

Turing Machines

Turing machine is the simplest model of computation.

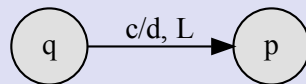
- Input written on (infinite) one sided tape.
- Special blank characters.
- Finite state control (similar to DFA).
- Every step: Read character under head, write character out, move the head right or left (or stay).
- Every TM M can be encoded as a string $\langle M \rangle$



Transition Function: $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow, \square\}$

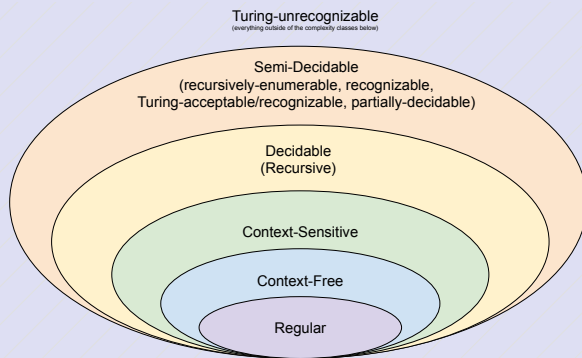
$\delta(q, c) = (p, d, \leftarrow)$

- q : current state.
- c : character under tape head.
- p : new state.
- d : character to write under tape head
- $\leftarrow$: Move tape head left.

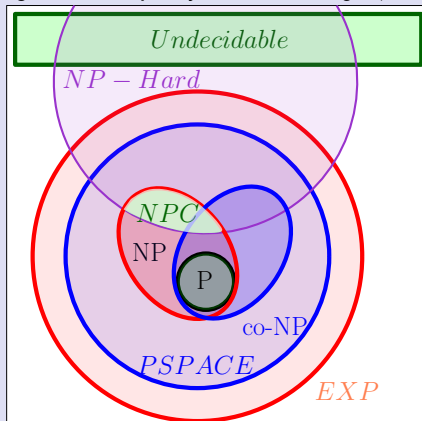


Complexity Classes

Computational Complexity Classes



Algorithmic Complexity Classes (assuming $P \neq NP$)



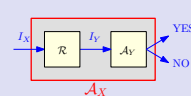
Reductions

A general methodology to prove impossibility results.

- Start with some *known* hard problem X
- Reduce X to your favorite problem Y

If Y can be solved then so can $X \implies Y$. But we know X is hard so Y has to be hard too. On the other hand if we know Y is easy, then X has to be easy too.

The Karp reduction, $X \leq_P Y$ suggests that there is a polynomial time reduction from X to Y .



Assuming

- $R(n)$: running time of R
 - $Q(n)$: running time of A_Y
- Running time of A_X is $O(Q(R(n)))$

Sample NP-complete problems

CIRCUITSAT: Given a boolean circuit, are there any input values that make the circuit output TRUE?

3SAT: Given a boolean formula in conjunctive normal form, with exactly three distinct literals per clause, does the formula have a satisfying assignment?

INDEPENDENTSET: Given an undirected graph G and integer k , what is there a subset of vertices $\geq k$ in G that have no edges among them?

CLIQUE: Given an undirected graph G and integer k , is there a complete subgraph of G with more than k vertices?

KPARTITION: Given a set X of kn positive integers and an integer k , can X be partitioned into n , k -element subsets, all with the same sum?

3COLOR: Given an undirected graph G , can its vertices be colored with three colors, so that every edge touches vertices with two different colors?

HAMILTONIANPATH: Given graph G (either directed or undirected), is there a path in G that visits every vertex exactly once?

HAMILTONIANCYCLE: Given a graph G (either directed or undirected), is there a cycle in G that visits every vertex exactly once?

LONGESTPATH: Given a graph G (either directed or undirected, possibly with weighted edges) and an integer k , does G have a path $\geq k$ length?

• Remember a **path** is a sequence of distinct vertices $[v_1, v_2, \dots, v_k]$ such that an edge exists between any two vertices in the sequence. A **cycle** is the same with the addition of an edge $(v_k, v_1) \in E$. A **walk** is a path except the vertices can be repeated.

• A formula is in conjunction normal form if variables are or'ed together inside a clause and then clauses are and'ed together: $((x_1 \vee x_2 \vee x_3) \wedge (\overline{x_2} \vee x_4 \vee x_5))$. Disjunctive normal form is the opposite $((x_1 \wedge x_2 \wedge x_3) \vee (\overline{x_2} \wedge x_4 \wedge x_5))$.

Sample undecidable problems

ACCEPTONINPUT: $A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts on } w \}$

HALTONINPUT: $Halt_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and halts on input } w \}$

HALTONBLANK: $Halt_{B_{TM}} = \{ \langle M \rangle \mid M \text{ is a TM \& } M \text{ halts on blank input} \}$

EMPTINESS: $E_{TM} = \{ \langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset \}$

EQUALITY: $EQ_{TM} = \left\{ \langle M_A, M_B \rangle \mid \begin{array}{l} M_A \text{ and } M_B \text{ are TM's} \\ \text{and } L(M_A) = L(M_B) \end{array} \right\}$